MSML610: Advanced Machine Learning

Lesson 09.5: Kalman Filter

Instructor: Dr. GP Saggese, gsaggese@umd.edu

References:

- AIMA 14: Probabilistic reasoning over time
- https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

- ***1-D Kalman Filters***
- Multivariate Kalman Filters
- Dynamic Bayesian networks

Kalman Filter: Intuition and Interpretations

- Extension of g–h filters
  - g–h uses fixed gains g, h for position and velocity
  - Kalman filter computes adaptive gain K_t from covariance P_t
  - Handles multidimensional states and general linear systems
- **State space model**
  - $x_t = Ax_{t-1} + w_t$
  - $y_t = Cx_t + v_t$
  - Gaussian noises with covariances Q, R
- **Bayesian interpretation**
  - Prior and likelihood Gaussian $\Rightarrow$ posterior Gaussian
  - Recursively updates mean and covariance
  - Example: radar tracking of aircraft
- **Orthogonal projection interpretation**
  - Estimate is projection onto span of observations
  - K_t minimizes $E[\|x_t - \hat{x}_t\|^2]$
  - Error orthogonal to measurement space

Updating Belief Using Gaussians

- The **Kalman filter** is based on Bayes theorem and Gaussians
- The **Bayes theorem** states:

$$\text{posterior} = \text{normalized}(\text{prior} \times \text{likelihood})$$

- If the *prior* and the *likelihood* are Gaussian the *posterior* is also Gaussian (conjugate prior)
  - Everything can be represented as Gaussians $\rightarrow$ encode the PDF in terms of mean and std dev
  - Updating belief is equivalent to sum and multiplication of Gaussians
- **Algorithm**

```
# Create prior (using current estimate and system model).
prior = predict(x, process_model)

# Create likelihood (using measurement).
likelihood = gaussian(z, sensor_var)

# Update belief using prior and likelihood
posterior = update(prior, likelihood)
```

Sum of Gaussians

- Given two two Gaussians X and Y

$$X \sim Normal(\mu_1, \sigma_1^2)$$
$$Y \sim Normal(\mu_2, \sigma_2^2)$$

- The sum $Z = X + Y$ is a Gaussian $Normal(\mu, \sigma^2)$ with:

$$\begin{cases} \mu = \mu_1 + \mu_2 \\ \sigma^2 = \sigma_1^2 + \sigma_2^2 \end{cases}$$

- **Interpretation:**
  - The mean is the sum of the mean (by linearity)
  - The variance always increases

Product of Gaussians

- Given two two Gaussians X and Y

$$X \sim Normal(\mu_1, \sigma_1^2)$$
$$Y \sim Normal(\mu_2, \sigma_2^2)$$

- The product $Z = X \cdot Y$ is a Gaussian $Normal(\mu, \sigma^2)$ with:

$$\begin{cases} \mu = \dfrac{\mu_1 \sigma_2^2 + \mu_2 \sigma_1^2}{\sigma_1^2 + \sigma_2^2} \\ \sigma^2 = \dfrac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2} \end{cases}$$

- **Interpretation:**
  - Reduce variance by incorporating more information
  - If one Gaussian X is narrower (more accurate), result leans towards X
  - If two Gaussians are similar (measures corroborate), result becomes more certain

Kalman Pseudo-Algorithm

- Typical symbols used in Kalman literature:

Symbol	Meaning
x	State
P	Variance of state (uncertainty, belief)
$f()$	System model
Q	System model error
z	Measurement
R	Measurement noise

- **Initialization**
  - Initialize state of filter $x = x_0$
  - Initialize belief in the state $P = P_0$

Kalman Pseudo-Algorithm

- **Predict stage**

```
def predict(posterior, movement):
  # Prior belief about state (mean x, variance P).
  x, P = posterior
  # System model: movement (mean dx) and its uncertainty Q.
  dx, Q = movement
  # Use system model to predict next state: x = f(x)
  x = x + dx
  # Adjust belief to account for uncertainty in prediction.
  P = P + Q
  return gaussian(x, P)
```

Kalman Pseudo-Algorithm

- **Update stage**

```
def update(prior, measurement):
  # Prior belief about state (mean x, variance P)
  x, P = prior
  # Measurement z and belief about its accuracy R.
  z, R = measurement
  # Compute residual between estimated state and measurement.
  y = z - x
  # Compute Kalman gain K from prediction uncertainty P and
  # measurement noise R.
  K = P / (P + R)
  # Update state estimate.
  x = x + K * y
  # Update belief (uncertainty).
  P = (1 - K) * P
  return gaussian(x, P)
```

Kalman Gain

- Assume that:
  - Prior prediction $x \sim Normal(\mu_x, \sigma_x^2)$
  - Measurement $z \sim Normal(\mu_z, \sigma_z^2)$
- The mean of the posterior (using the multiplication expression) is:

$$\mu = \frac{\sigma_x^2 \mu_z + \sigma_z^2 \mu_x}{\sigma_x^2 + \sigma_z^2} = \frac{\sigma_x^2}{\sigma_x^2 + \sigma_z^2}\mu_z + \frac{\sigma_z^2}{\sigma_x^2 + \sigma_z^2}\mu_x = K\mu_z + (1-K)\mu_x$$

- The Kalman Gain $K = \dfrac{\sigma_x^2}{\sigma_x^2 + \sigma_z^2}$:
  - Is the scaling term that mixes the prediction and the measurement
  - Depends on the ratio of uncertainty of prior and measurement

- 1-D Kalman Filters
- ***Multivariate Kalman Filters***
  - Tracking a Dog with a Kalman Filter
  - Non-Linear Filtering
- Dynamic Bayesian networks

Multivariate State

- Often the state variable is multivariate, e.g.,
  - Position and velocity of a dog (probably uncorrelated)
  - Height and weight of an adult (correlated)
- **Variance** is a measure of how a population varies, e.g.,
  - Variance $= 0$ means constant
  - Large variance means lots of variation
- **Covariance** are correlated variances
  - E.g., height and weight are generally positively correlated
- **Covariance matrix**
  - The diagonal contains the variance for each variable
  - The off-diagonal elements contain the covariance between i and j variables
  - The covariance matrix is symmetric
- Correlation allows prediction
  - E.g., "as winter comes you predict you will spend more on heating your house"

Multivariate Gaussian

- The marginal of a multivariate Gaussian is 1-d Gaussian
- Consider a contour plot (i.e., the intersection of a 2-d Gaussian $z = f(x, y)$ with a plane $z = c$)
  - The contour plot is always an ellipses

Multiplying Two Multivariate Gaussians

- Given two multivariate Gaussians $\sim Normal(\underline{\boldsymbol{\mu}}_i, \underline{\underline{\boldsymbol{\Sigma}}}_i)$
- The product of the Gaussians is still Gaussian $\sim Normal(\underline{\boldsymbol{\mu}}, \underline{\underline{\boldsymbol{\Sigma}}})$

$$\underline{\boldsymbol{\mu}} = \underline{\underline{\boldsymbol{\Sigma}}}_2(\underline{\underline{\boldsymbol{\Sigma}}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_2)^{-1}\underline{\boldsymbol{\mu}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_1(\underline{\underline{\boldsymbol{\Sigma}}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_2)^{-1}\underline{\boldsymbol{\mu}}_2$$
$$\underline{\underline{\boldsymbol{\Sigma}}} = \underline{\underline{\boldsymbol{\Sigma}}}_1(\underline{\underline{\boldsymbol{\Sigma}}}_1 + \underline{\underline{\boldsymbol{\Sigma}}}_2)^{-1}\underline{\underline{\boldsymbol{\Sigma}}}_2$$

- **Note**: this is a generalization of the 1-d case to multivariate

$$\mu = \frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$
$$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

replacing:

- σ^2 with covariance matrix $\underline{\underline{\boldsymbol{\Sigma}}}$
- Division with matrix inversion

Multivariate Filtering

- Covariance structure helps improve the estimate, e.g.,
  - You know an airplane direction can't change quickly
  - Knowing an approximate value for the velocity helps constrain possible next positions
- **E.g., airplane**
  - You are tracking a plane moving in a direction (1-d problem)
  - At time 1, you are fairly certain about the position $x = 0$, but you don't know the velocity
    - You plot position and velocity on an x-y plane
    - The covariance matrix between position and velocity is narrow and tall
    - It is narrow on the x-axis since you know that the position is around $x = 0$
    - It is tall on the y-axis because of your lack of knowledge about velocity
  - After 1 sec, you get a position update of $x = 5$
    - You can infer that the velocity is 5/s
    - The covariance matrix is then stretched diagonally

Notation

- A Bayesian notation $a|b$ means "a given the evidence of b"
  - The prior is $\underline{\hat{\boldsymbol{x}}}_{t|t-1}$, since you know only the information at time $t-1$, i.e., the previous state
  - The posterior is $\underline{\hat{\boldsymbol{x}}}_{t|t}$, since you know all the information at time t, i.e., the measurement
- **A simpler notation:**
  - Indicate the "prior" version of the variables (i.e., after the system update) with an overline (E.g., $\overline{x}$, $\overline{\boldsymbol{x}}$, $\overline{\boldsymbol{X}}$)
  - Omit the indices $t+1$ and t and use an assignment notation (representing "update in place" of a variable):

$$x = x + 1$$

    instead of the mathematical notation using a different variable for each time step:

$$x_{t+1} = x_t + 1$$

  - With this notation:
    - The prior is $\overline{x} = \hat{x}_{t|t-1}$
    - The posterior is $x = \hat{x}_{t|t}$

SCIENCE ACADEMY

Multivariate Kalman Filter

- With the previous notation:
  - State update: $\overline{\boldsymbol{x}} = \boldsymbol{F}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}$
  - State uncertainty: $\overline{\boldsymbol{P}} = \boldsymbol{F}\boldsymbol{P}\boldsymbol{F}^T + \boldsymbol{Q}$
  - Residual: $\boldsymbol{y} = \boldsymbol{z} - \boldsymbol{H}\overline{\boldsymbol{x}}$
  - Kalman gain: $\boldsymbol{K} = \overline{\boldsymbol{P}}\boldsymbol{H}^T(\boldsymbol{H}\overline{\boldsymbol{P}}\boldsymbol{H}^T + \boldsymbol{R})^{-1}$
  - Updated state: $\boldsymbol{x} = \overline{\boldsymbol{x}} + \boldsymbol{K}\boldsymbol{y}$
  - Update state uncertainty: $\boldsymbol{P} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\overline{\boldsymbol{P}}$
- Where
  - $\boldsymbol{x}$ and $\boldsymbol{P}$ are the state mean and covariance
  - $\boldsymbol{F}$ is the state transition function
  - $\boldsymbol{Q}$ is the system error (i.e., the noise in the model assessment)
  - $\boldsymbol{B}$ and $\boldsymbol{u}$ model the control inputs to the system
  - $\boldsymbol{H}$ is the measurement function
  - $\boldsymbol{z}$ and $\boldsymbol{R}$ are the measurement mean and covariance
  - $\boldsymbol{y}$ is the residual
  - $\boldsymbol{K}$ is the Kalman gain
- Use the system model to predict the next state
  - When we multiply $\boldsymbol{F}$ to $\boldsymbol{x}$ we get the prior (i.e., the state before seeing any measurement)
- Form an estimate between the prior and the measurement

From Univariate to Multivariate Kalman Filter

- Let's compare

Definition	Univariate (Bayesian)	Univariate (Kalman)	Multivariate (Kalman)
State update	$\overline{\mu} = \mu + \mu_f$	$\overline{x} = x + dx$	$\overline{\boldsymbol{x}} = \boldsymbol{F}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}$
State uncertainty	$\overline{\sigma}^2 = \sigma^2 + \sigma_f^2$	$\overline{P} = P + Q$	$\overline{\boldsymbol{P}} = \boldsymbol{F}\boldsymbol{P}\boldsymbol{F}^T + \boldsymbol{Q}$
Residual		$y = z - \overline{x}$	$\boldsymbol{y} = \boldsymbol{z} - \boldsymbol{H}\overline{\boldsymbol{x}}$
Kalman gain		$K = \frac{\overline{P}}{\overline{P}+R}$	$\boldsymbol{K} = \overline{\boldsymbol{P}}\boldsymbol{H}^T(\boldsymbol{H}\overline{\boldsymbol{P}}\boldsymbol{H}^T + \boldsymbol{R})^{-1}$
Updated state	$\hat{\mu} = \frac{\overline{\sigma}^2\mu_z + \sigma_z^2\overline{\mu}}{\overline{\sigma}^2 + \sigma_z^2}$	$x = \overline{x} + Ky$	$\boldsymbol{x} = \overline{\boldsymbol{x}} + \boldsymbol{K}\boldsymbol{y}$
Upd. state uncertainty	$\sigma^2 = \frac{\overline{\sigma}^2\sigma_z^2}{\overline{\sigma}^2 + \sigma_z^2}$	$P = (1 - K)\overline{P}$	$\boldsymbol{P} = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\overline{\boldsymbol{P}}$

SCIENCE ACADEMY

Designing a Kalman filter

- The designer of the model needs to design:
  - The form of the state $\underline{\boldsymbol{x}}$ and $\underline{\underline{\boldsymbol{P}}}$
  - The system model $\underline{\underline{\boldsymbol{F}}}$ and $\underline{\underline{\boldsymbol{Q}}}$
  - The measurement $\underline{\boldsymbol{z}}$ and $\underline{\underline{\boldsymbol{R}}}$
  - The measurement function $\underline{\underline{\boldsymbol{H}}}$
  - The control inputs $\underline{\underline{\boldsymbol{B}}}$ and $\underline{\boldsymbol{u}}$ if there are control inputs

- 1-D Kalman Filters
- Multivariate Kalman Filters
  - ***Tracking a Dog with a Kalman Filter***
  - Non-Linear Filtering
- Dynamic Bayesian networks

Tracking 1D Dog: Problem formulation

- There is a dog moving on a 1-d track
- The dog moves approximately 1 meter per step
  - The velocity has variance due to noise/imperfect model specification
- There is a sensor that measures the position of the dog
  - The sensor has a certain error
- Time is discrete

(Nuvolo)

Tracking Dog: Predict Step

- At each step, the position is described with a Gaussian distribution $Normal(\mu, \sigma^2)$
- The position is part of the system's state, along with the velocity
  - The position is "observed" by a sensor
  - The velocity is a "hidden" variable
  - You could use more variables (E.g., acceleration, jerk, etc.)

Tracking Dog: Design State Covariance

- Initialize variances to reasonable values
  - E.g., $\sigma_{position} = 500m$ due to uncertainty about initial position
  - Top speed for a dog is 21m/s, so set $3\sigma_{velocity} = 21$
  - Assume covariances to be zero due to unknown initial correlation between position and velocity
  - $\underline{\underline{\boldsymbol{P}}}$ is diagonal

Tracking Dog: Design System Model

- Describe mathematically the behavior of the system

$$x_{t+1} = x_t + v\Delta t$$

- No model to predict how dog velocity changes over time
  - Assume it remains constant

$$\dot{x}_{t+1} = \dot{x}_t$$

  - This is not correct, but if velocity doesn't change much, the filter will perform well
- Put the model in matrix form $\underline{\boldsymbol{x}}_{t+1} = \underline{\underline{\boldsymbol{F}}}\underline{\boldsymbol{x}}_t$

Tracking Dog: Predicting the System

- If we predict the system without measurements:
  - The state follows the system model
  - The state uncertainty grows
    - This is true even without system error (noise)

Tracking Dog: Design System Noise

- Consider a car driving on a road with cruise control on
- It should travel at constant speed:

$$x_t = \dot{x}_{t-1}\Delta t + x_{t-1}$$

- In reality, it is affected by unknown factors:
  - The cruise control is not perfect
  - Wind, hills, potholes affect the car
  - Passengers roll down windows, changing the drag profile of the car
- Model this as:

$$\dot{x}_t = \dot{x}_{t-1} + w$$

- Model all of this with a covariance matrix $\underline{\underline{\boldsymbol{Q}}} = \mathbb{E}[\underline{\boldsymbol{w}} \cdot \underline{\boldsymbol{w}}^T]$:
  - Assume the noise is iid, has zero mean, and is independent from the system
  - For these reasons, you don't have to change the position, only the velocity

Tracking Dog: Design the Control Function

- Incorporate control inputs to predict state based on this information

$$\Delta \underline{\overline{\boldsymbol{x}}} = \underline{\underline{\boldsymbol{B}}\boldsymbol{u}}$$

- E.g., in the case of the car
  - Steering
  - Acceleration
- E.g., in the case of the dog, control inputs can be
  - The voice of its master
  - Seeing a squirrel

Tracking Dog: Design the Measurement Function

- Kalman filter computes the update step in the measurement space
- If the measurement is in the same units as the state, the residual is simple to compute:

$$\text{residual} = \text{measured position - predicted position}$$

- E.g., assume we are tracking the position of the dog using a sensor that outputs a voltage
  - We cannot compute the residual as:

$$\text{measure voltage - predicted position}$$

  - We need to convert the position into voltage
- The Kalman space allows to have a measurement matrix $\underline{\underline{\boldsymbol{H}}}$ to convert the state into a measurement

$$\underline{\boldsymbol{y}} = \underline{\boldsymbol{z}} - \underline{\underline{\boldsymbol{H}}}\overline{\boldsymbol{x}}$$

Why Working in Measurement and Not in State Space?

- The problem is that it is possible to convert state into measurement, but not vice versa because of the hidden variables
  - E.g., transform position (discarding velocity) into voltage
  - If the sensor doesn't read velocity how do we estimate the measured velocity

Tracking Dog: Design the Measurement

- Typically $\underline{\boldsymbol{z}}$ is easy since it just contains the measurements from the sensor
- The measurement noise matrix $\underline{\underline{\boldsymbol{R}}}$ can be difficult to estimate
  - Noise can be not Gaussian
  - There can be a bias in the sensor
  - The error can be not symmetrical (e.g., temperature sensor is less precise as the temperature increases)

- 1-D Kalman Filters
- Multivariate Kalman Filters
  - Tracking a Dog with a Kalman Filter
  - ***Non-Linear Filtering***
- Dynamic Bayesian networks

Optimality

- **Assumptions:**
  - Everything is linear
  - System and sensor noise is Gaussian
- Under these assumptions, the Kalman filter is optimal in a least square sense
- The Kalman filter is a mathematical model of the world
  - The output is only as accurate as the model of the world

The World Is Non-Linear

- The Kalman filter uses linear equations and can only handle linear problems
- **The world is non-linear:**
  - System model can be non-linear:
    - Many physical systems are described by non-linear differential equations
    - E.g., a ball flying through air is affected by drag, leading to non-linear behaviors
  - Measurements can be non-linear:
    - To measure the height on a plane, you can measure the distance of the plane from the radar. Given the Pythagorean theorem, you get:

$$x = \sqrt{\text{dist}^2 - \text{height}^2}$$

- Rarely does a physical system have equations that can be solved analytically

Extended Kalman Filter

- Aka EKF
- EKF is a nonlinear version of the Kalman filter
  - Linearize the differential equations to compute the Jacobian (i.e., matrix of partial derivatives) at the point of the current estimate
  - Used for estimating the state of a nonlinear dynamic system
- Pros
  - Use the linear Kalman machinery
- Cons:
  - Analytical solution:
    - Difficult or impossible
  - Numerical solution:
    - Expensive computationally
    - Errors can compound forcing the filter to diverge (unstable)

Unscented Kalman Filter

- Aka UKF
- It is superior to EKF in almost every way

Intuition of Sampling Techniques

- Assume you have a distribution X and a non-linearity ϕ
- For every measurement:
  - Generate many points from X
  - Pass them through the non-linear function ϕ
  - Approximate the result (E.g., compute mean and variance)
- **Problem**:
  - "How many points are needed to build an accurate output distribution"?
  - Even if n=500,000 points are enough for 1 dimension, for k dimensions you might need n^k points (curse of dimensionality)

Unscented Transform

- Unscented transform estimates the result of applying a non-linear transformation to a probability distribution characterized by a finite number of moments (e.g., mean and covariance)
  - E.g., compute the non-linear transform of a distribution, given mean and covariance estimate
  - Called "unscented" since "it doesn't stink."
- **Intuition**
  - Given a PDF C with mean $\underline{\boldsymbol{\mu}}$ and covariance $\underline{\underline{\boldsymbol{\Sigma}}}$
  - Encode mean and covariance in a set of points (sigma points) that represent a discrete PMF D with the same mean $\underline{\boldsymbol{\mu}}$ and covariance $\underline{\underline{\boldsymbol{\Sigma}}}$
  - Propagate the discrete PMF D by applying the non-linear function ϕ to each point of the PMF
  - The mean and covariance of $\phi(D)$ approximate the mean and covariance of $\phi(C)$

Unscented Transform: 1D Case

- The idea is that we need 3 sigma points for a 1-d Gaussian
  - One point for the mean
  - Two points around the mean
- Each point has a weight

Unscented Transform: Sigma Points

- Consider a distribution F and a non-linearity ϕ
- There are algorithms to generate points and weights (given the mean and covariance of F) to evaluate mean and covariance of F transformed through ϕ
- In n dimensions, we need $2n+1$ points $\underline{\boldsymbol{x}}_i$ and weights w_i^m, w_i^c

$$\sum_i w_i^m = 1$$

$$\sum_i w_i^c = 1$$

$$\mu(\phi) = \sum_i w_i^m \phi(\underline{\boldsymbol{x}}_i)$$

$$\Sigma(\phi) = \sum_i w_i^c (\phi(\underline{\boldsymbol{x}}_i) - \mu(\phi))(\phi(\underline{\boldsymbol{x}}_i) - \mu(\phi))^T$$

- Note that selecting the sigma points has not a single solution

Monte Carlo Sampling

- Use a finite number of randomly sampled points to represent the problem
- Run the points through the transformation (e.g., non-linear function / system you are modeling)
- Compute the results on the transformed points

Particle Filters

- Aka Sequential Monte Carlo (SMC) methods
- = Monte Carlo algorithms to solve problems in Bayesian statistical inference (e.g., in filtering problems)
- The goal is to compute posterior distributions of the states, given some data

Generic particle filter algorithm

1. Randomly generate particle

- Particles have all state variable that needs to be estimated (e.g., position, velocity)
- Each particle has a weight representing the probability that it represents the actual state of the system

2. Predict next state of the particles
3. Update weighting

- Update the weighting of the particles based on the measurements
- Particles that match closely the measurements are weighted higher

4. Resample

- Discard highly improbable particle

5. Compute estimate

- Compute weighted mean and covariance of the particles to get an estimate of the state and uncertainty

- 1-D Kalman Filters
- Multivariate Kalman Filters
- ***Dynamic Bayesian networks***

Dynamic Bayesian Networks (DBNs)

- DBNs extend Bayesian networks to model temporal processes
- Main idea
  - "Unroll" the model over time
  - Capture intra-slice (within time) and inter-slice (across time) dependencies
- Each time slice includes:
  - State variables X_t
  - Evidence variables E_t
- Assumptions
  - First-order Markov process: current state depends only on the previous state
  - First-order sensor Markov process: evidence depends only on current state
  - Stationarity: each time slice is the same, both structure and parameters do not change over time
    - Structure and CPTs (Conditional Probability Tables) are the same across slices (time-homogeneous model)
  - No Gaussian distribution

DBNs vs HMMs

- DBNs generalize Hidden Markov Models (HMMs)
- HMMs are a special case with a single hidden and evidence variable per time step
- DBNs model more complex systems than HMMs by:
  - Using multiple state variables
    - Enables modeling large systems like robot localization with many state components
  - Exploiting sparse connections among variables yielding compact model
    - HMM: transition matrix of size $O(d^{2n})$
    - DBN: size $O(nd^k)$ with k bounded parents per variable

DBNs vs Kalman Filters

- DBNs generalize Kalman filters
- Every Kalman filter can be represented in a DBN with:
  - Continuous variables
  - Linear / Gaussian conditional distributions
- Not every DBN can be represented by a Kalman filter, since:
  - DBN variables can mix discrete/continuous and non-Gaussian
  - Allow arbitrary conditional dependencies among variables
- **Pros of DBNs**
  - DBNs are applicable to broader domains including:
    - Fault diagnosis in networks
    - Complex system monitoring
- **Pros of Kalman filters:**
  - Optimal for linear systems with Gaussian noise
  - Support exact inference, DBNs often require approximate methods

Constructing a DBN

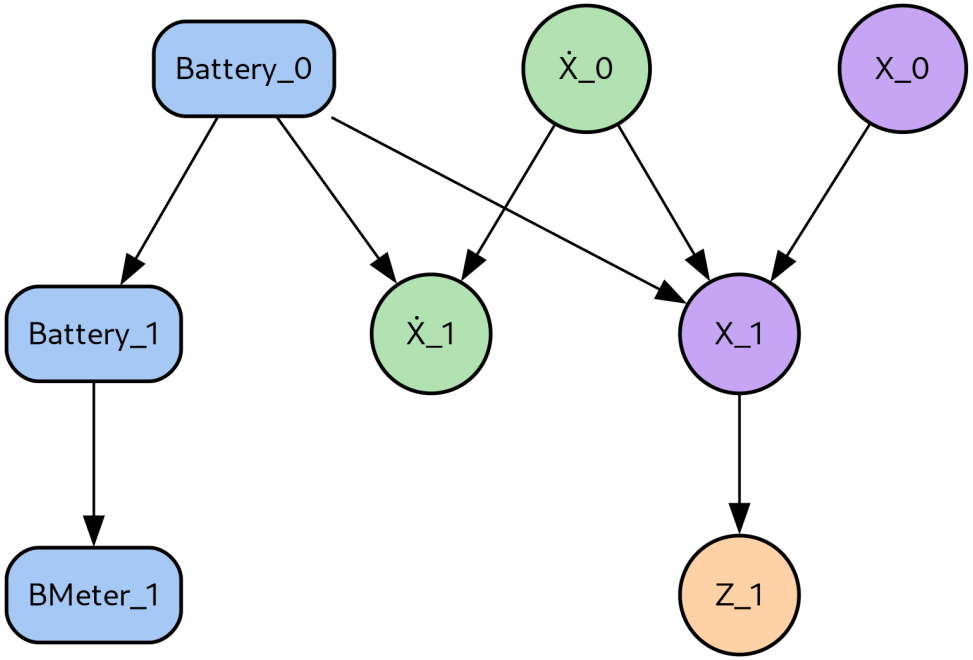
- Key components of a DBN
  - Prior distribution of state $\Pr(X_0)$
  - Transition model $\Pr(X_{t+1}|X_t)$
  - Sensor model $\Pr(E_t|X_t)$
  - Transition and sensor models are time-homogeneous
- Network topology includes:
  - Intra-slice topology
  - Inter-slice links

DBN Example: Tracking a Robot (1/3)

- **Problem:**
  - Tracking a robot moving randomly on a line X over time
- **Initial model:**
  - Position X_t and velocity $\dot{X}_t$ as state variables
  - Update via Newton's laws
  - Easy to generalize for 2d or 3d by using a $\underline{\boldsymbol{X}}_t$
- **Issue:**
  - Velocity changes over time
  - Battery exhaustion affects velocity systematically
  - Effect depends on cumulative energy use
  - Violates the Markov property (future depends on full history)
- **Solution:**
  - Include battery level $Battery_t$ in the state X_t
  - Restores the Markov assumption
  - Allows motion prediction considering energy constraints
  - Enables coherent reasoning about motion and power consumption over time
- **New requirement for state:**
  - $S_t = (X_t, \dot{X}_t, \text{BatteryLevel}_t)$
  - $E_t = (\text{GPS}_t, \text{BMeter}_t)$

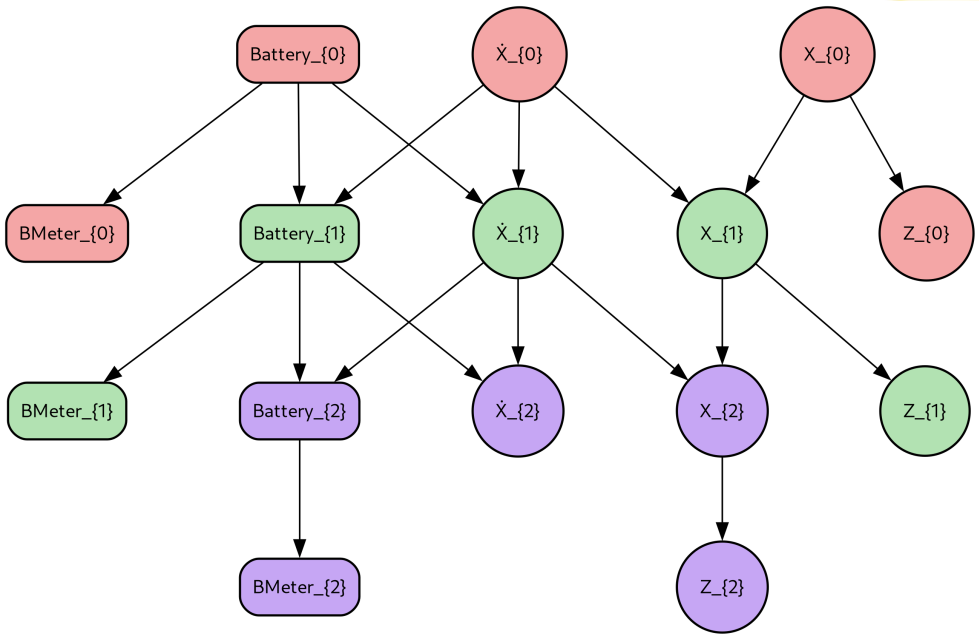
DBN Example: Tracking a Robot (2/3)

- The DBN structure models both intra-slice (within time) and inter-slice (across time) dependencies
- Intra-slice dependencies:
  - Position X_t influences velocity $\dot{X}_t$
  - BatteryLevel_t influences velocity $\dot{X}_t$
  - Battery_{t+1} depends on Battery_t and $\dot{X}_t$
  - BMeter_t depends on Battery_t
  - GPS_t depends on X_t
- Inter-slice dependencies:
  - Position X_{t+1} depends on Position X_t and velocity $\dot{X}_t$
  - Velocity $\dot{X}_{t+1}$ depends on $\dot{X}_t$ and Battery_t



DBN Example: Tracking a Robot (3/3)

- **Replicate for Multiple Time Slices**:
  - Create slices for $t = 0, 1, 2, \ldots$ with the above variables and dependencies
  - Group each time slice vertically or horizontally for clarity
- **Unrolling**:
  - Visualize the full DBN by unrolling these slices over the desired number of time steps (e.g., three slices for $t = 0, 1, 2$)



Inference in DBNs

- DBNs are Bayesian networks and we can use the same inference algorithms
  - "Unroll" the DBN over time (i.e., replicate slices for each time step) and apply standard BN inference
  - We can't unroll "forever", but we limit to a certain number of slices to approximate a fixed amount of time dependency
- Use recursive methods to get a constant time and space update complexity
  - Variable elimination with temporal ordering
  - At time step $t+1$ add slice $t+2$ and remove slice t so one has always two slices to do inference
  - Maintains constant memory by keeping only two slices at a time
- Complexity:
  - Exponential in number of state variables ($O(nd^{n+k})$)
  - More efficient than full HMM representation ($O(d^{2n})$)
- Even though we can use DBNs to represent very complex temporal processes with many sparsely connected variables, we cannot reason efficiently and exactly about those processes
  - The prior joint distribution over all the variables is factorizable into its constituents CPTs
  - The posterior joint distribution conditioned on observation sequence is not

Approximate Inference in DBNs

- Particle Filtering:
  - Represent belief state with weighted samples (particles)
  - Steps: propagate, weight, resample
- Benefits:
  - Focuses computation on high-probability regions
  - Maintains manageable memory and time per step
- Challenges:
  - Approximation error
  - Sensitive to transition and observation model assumptions
- Used when exact inference is computationally impractical
- Real-world application: robot localization, speech recognition

DBN to Represent Changing Model

- We can model the fact that the system can change over time
  - Transient failure: a sensor reads wrong measures
  - Persistent failure model: we can model it with additional variables (e.g., *SensorBroken*)

DBN: Inference

- We can unroll the DBN and get a BayesNet and then perform exact or approximate inference with the known methods (e.g., MCMC)

DBN: Optimization for Inference

- Many optimizations are possible, e.g.,
  - Instead of running each sample through the entire DBN one can run all the samples evaluating one slice at a time to compute the posterior distribution